In [12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib as mpl

mpl.rcParams["font.enable_last_resort"] = False
```

Data load

In [3]:
```
df = pd.read_csv("Pakistan Largest Ecommerce Dataset.csv", low_memory=False)
print("Shape:", df.shape)
print(df.columns.tolist())
df.head()
```

```
Shape: (1048575, 26)
['item_id', 'status', 'created_at', 'sku', 'price', 'qty_ordered', 'grand_total', 'i
ncrement_id', 'category_name_1', 'sales_commission_code', 'discount_amount', 'paymen
t_method', 'Working Date', 'BI Status', ' MV ', 'Year', 'Month', 'Customer Since',
'M-Y', 'FY', 'Customer ID', 'Unnamed: 21', 'Unnamed: 22', 'Unnamed: 23', 'Unnamed: 2
4', 'Unnamed: 25']
```

Out[3]:

	item_id	status	created_at	sku	price	qty_ordered	grand_total	in
0	211131.0	complete	7/1/2016	kreations_YI 06-L	1950.0	1.0	1950.0	
1	211133.0	canceled	7/1/2016	kcc_Buy 2 Frey Air Freshener & Get 1 Kasual Bo...	240.0	1.0	240.0	
2	211134.0	canceled	7/1/2016	Ego_UP0017-999-MR0	2450.0	1.0	2450.0	
3	211135.0	complete	7/1/2016	kcc_krone deal	360.0	1.0	60.0	
4	211136.0	order_refunded	7/1/2016	BK7010400AG	555.0	2.0	1110.0	

5 rows × 26 columns

In [8]:
```
df.describe()
```

Out[8]:

	item_id	created_at	price	qty_ordered	grand_total	disc
count	584524.000000	584524	5.845240e+05	584524.000000	5.845240e+05	5
mean	565667.074218	2017-08-08 11:42:53.589450752	6.348748e+03	1.296388	8.530619e+03	
min	211131.000000	2016-07-01 00:00:00	0.000000e+00	1.000000	-1.594000e+03	
25%	395000.750000	2017-01-29 00:00:00	3.600000e+02	1.000000	9.450000e+02	
50%	568424.500000	2017-08-17 00:00:00	8.990000e+02	1.000000	1.960400e+03	
75%	739106.250000	2018-02-03 00:00:00	4.070000e+03	1.000000	6.999000e+03	
max	905208.000000	2018-08-28 00:00:00	1.012626e+06	1000.000000	1.788800e+07	
std	200121.173648	NaN	1.494927e+04	3.996061	6.132081e+04	

Missing values Check

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
item_id                  464051
status                   464066
created_at               464051
sku                      464071
price                    464051
qty_ordered              464051
grand_total              464051
increment_id             464051
category_name_1          464215
sales_commission_code    601229
discount_amount          464051
payment_method           464051
Working Date             464051
BI Status                464051
 MV                      464051
Year                     464051
Month                    464051
Customer Since           464062
M-Y                      464051
FY                       464051
Customer ID              464062
Unnamed: 21             1048575
Unnamed: 22             1048575
Unnamed: 23             1048575
Unnamed: 24             1048575
Unnamed: 25             1048575
dtype: int64
```

Data Cleaning

Remove empty columns empty rows and duplicates

In [5]:
```
df = df.loc[:, ~df.columns.str.contains("Unnamed")]
df = df.dropna(how="all")

df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
df = df.rename(columns={"increment_id": "order_id"})

df = df.drop_duplicates()
print(df.shape)
```

```
(584524, 21)
```

Fix data types

In [6]:
```
df["created_at"] = pd.to_datetime(df["created_at"], errors="coerce")

for col in ["price", "qty_ordered", "grand_total", "discount_amount"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df.dtypes
```

Out[6]:
```
item_id                       float64
status                         object
created_at             datetime64[ns]
sku                            object
price                         float64
qty_ordered                   float64
grand_total                   float64
order_id                       object
category_name_1                object
sales_commission_code          object
discount_amount               float64
payment_method                 object
working_date                   object
bi_status                      object
mv                             object
year                          float64
month                         float64
customer_since                 object
m-y                            object
fy                             object
customer_id                   float64
dtype: object
```

Clean text columns

In [7]:
```
for col in ["status", "payment_method", "category_name_1"]:
    df[col] = df[col].astype(str).str.strip().str.lower()
    df[col] = df[col].replace(["\\n", "nan", ""], np.nan)

print(df["status"].value_counts(), "\n")
print(df["payment_method"].value_counts(), "\n")
print(df["category_name_1"].value_counts())
```

```
status
complete          233685
canceled          201249
received           77290
order_refunded     59529
refund              8050
cod                 2859
paid                1159
closed               494
payment_review        57
pending               48
processing            33
holded                31
fraud                 10
pending_paypal         7
exchange               4
Name: count, dtype: int64

payment_method
cod                  271960
payaxis               97641
easypay               82900
jazzwallet            35145
easypay_voucher       31176
bankalfalah           23065
jazzvoucher           15633
easypay_ma            14028
customercredit         7555
apg                    1758
ublcreditcard           882
cashatdoorstep          732
mcblite                 723
mygateway               669
internetbanking         472
productcredit           125
marketingexpense         45
financesettlement        15
Name: count, dtype: int64

category_name_1
mobiles & tablets     115710
men's fashion          92221
women's fashion        59721
appliances             52413
superstore             43613
beauty & grooming      41496
soghaat                34011
others                 29218
home & living          26504
entertainment          26326
health & sports        17502
kids & baby            16494
computing              15933
school & education      3478
books                   1870
Name: count, dtype: int64
```

Group status and payment method

In [9]:
```
status_map = {
    "complete": "Completed",
    "received": "Completed",
    "canceled": "Cancelled",
    "order_refunded": "Refunded",
    "refund": "Refunded",
    "closed": "Refunded",
}
df["status_clean"] = df["status"].map(status_map).fillna("Other")

# payment type: COD, Prepaid, or Store Credit
def get_payment_type(x):
    if x in ["cod", "cashatdoorstep"]:
        return "COD"
    if x in ["customercredit", "productcredit", "marketingexpense", "financesettlem
        return "Store Credit"
    return "Prepaid"

df["payment_type"] = df["payment_method"].apply(get_payment_type)

print(df["status_clean"].value_counts())
print(df["payment_type"].value_counts())
```

```
status_clean
Completed    310975
Cancelled    201249
Refunded      68073
Other          4227
Name: count, dtype: int64
payment_type
Prepaid         304092
COD             272692
Store Credit      7740
Name: count, dtype: int64
```

Remove missing and invalid rows

In [10]:
```
# drop rows where important columns are empty
df = df.dropna(subset=["status", "payment_method", "category_name_1",
                       "price", "qty_ordered", "created_at", "customer_id"])

# drop invalid rows (zero or negative price/quantity)
df = df[(df["price"] > 0) & (df["qty_ordered"] > 0)]

# create our own total column
df["item_total"] = df["price"] * df["qty_ordered"]
print(df.shape)
```

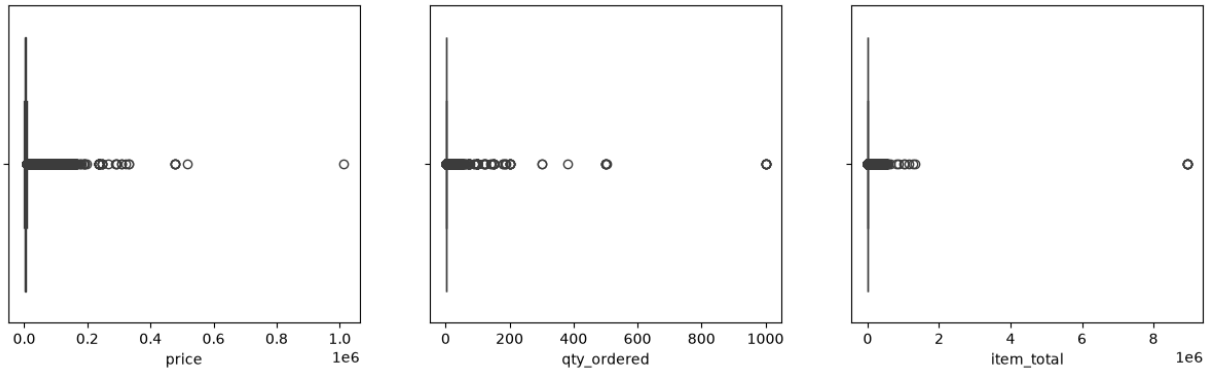
```
(574281, 24)
```

Outlier Handling

Mobile phones and laptops are expensive, and these are genuine orders, so we do not remove all outliers. Instead, we remove only the top 0.1% of extreme values.

In [14]:
```
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
sns.boxplot(x=df["price"], ax=ax[0])
sns.boxplot(x=df["qty_ordered"], ax=ax[1])
sns.boxplot(x=df["item_total"], ax=ax[2])
plt.show()

df[["price", "qty_ordered", "item_total"]].describe(percentiles=[0.99, 0.999])
```



Out[14]:

	price	qty_ordered	item_total
count	5.742810e+05	574281.000000	5.742810e+05
mean	6.405107e+03	1.292470	7.245600e+03
std	1.503044e+04	3.982037	3.422371e+04
min	1.000000e-01	1.000000	1.000000e-01
50%	9.200000e+02	1.000000	9.990000e+02
99%	8.377000e+04	5.000000	8.604200e+04
99.9%	1.180000e+05	12.000000	2.119980e+05
max	1.012626e+06	1000.000000	8.944000e+06

Feature Engineering

In [15]:
```
# date features
df["year"] = df["created_at"].dt.year
df["month"] = df["created_at"].dt.month
df["year_month"] = df["created_at"].dt.strftime("%Y-%m")

# flag columns (1 or 0)
df["is_cancelled"] = (df["status_clean"] == "Cancelled").astype(int)
```

```
df["is_refunded"] = (df["status_clean"] == "Refunded").astype(int)
df["is_failed"] = df["is_cancelled"] + df["is_refunded"]
df["is_final"] = (df["status_clean"] != "Other").astype(int)

# discount percentage and bands
df["discount_pct"] = (df["discount_amount"] / df["item_total"] * 100).clip(0, 100)
df["discount_band"] = pd.cut(df["discount_pct"], [-1, 0, 10, 25, 50, 100],
                             labels=["No discount", "1-10%", "11-25%", "26-50%", "5

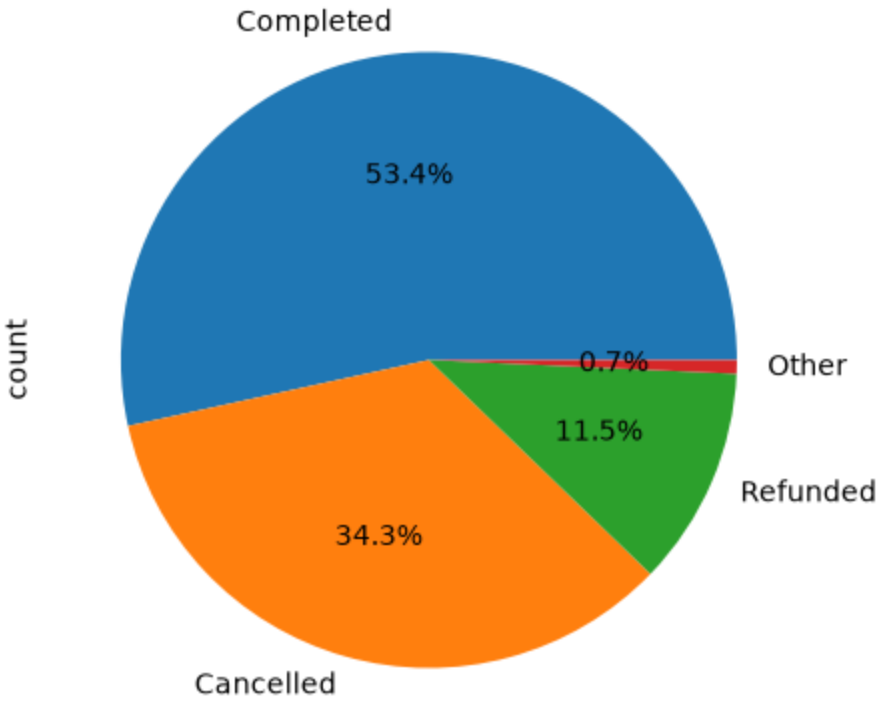
# order value bands
df["value_band"] = pd.qcut(df["item_total"], 4, labels=["Low", "Medium", "High", "P

# net revenue (only completed orders)
df["net_revenue"] = np.where(df["status_clean"] == "Completed", df["item_total"], 0
```

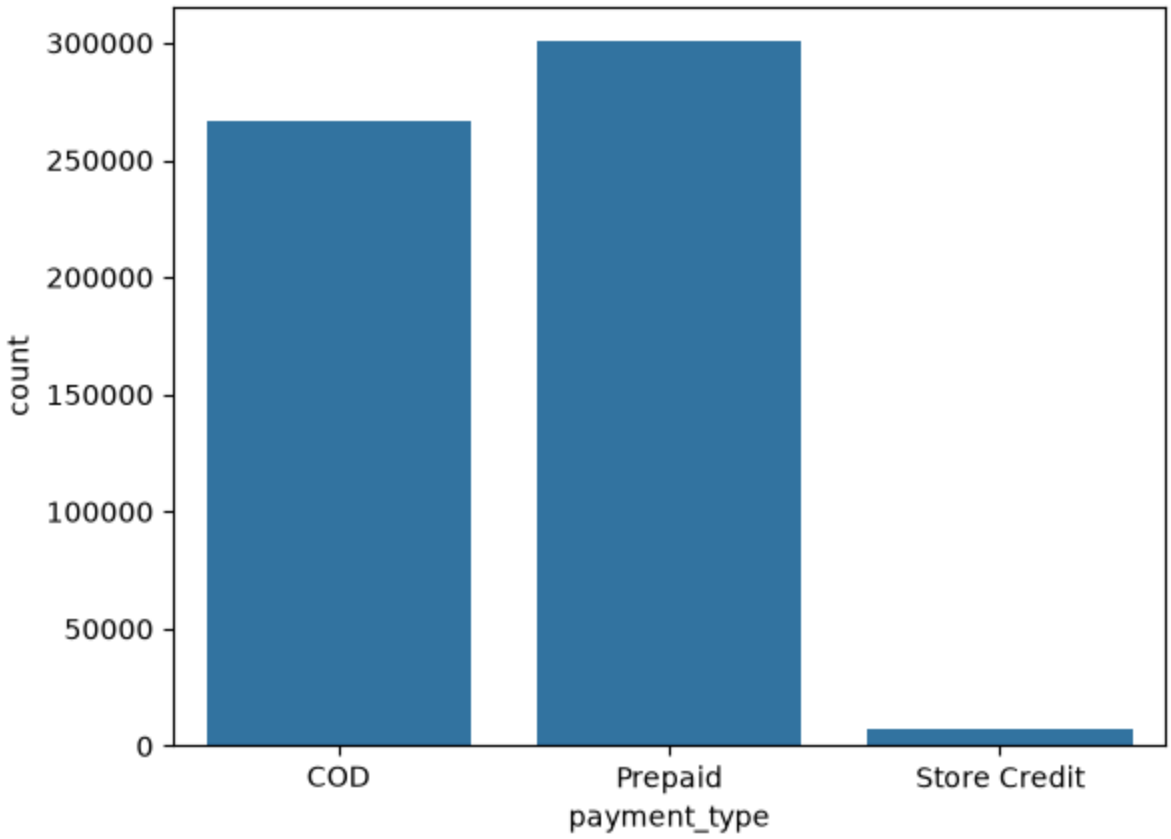
Exploratory Data Analysis (EDA)

The failed rate is calculated only for final orders (Completed, Cancelled, and Refunded).

In [18]:
```
df["status_clean"].value_counts().plot.pie(autopct="%1.1f%%", figsize=(5, 5))
plt.show()
```



In [19]:
```
sns.countplot(data=df, x="payment_type")
plt.show()
```



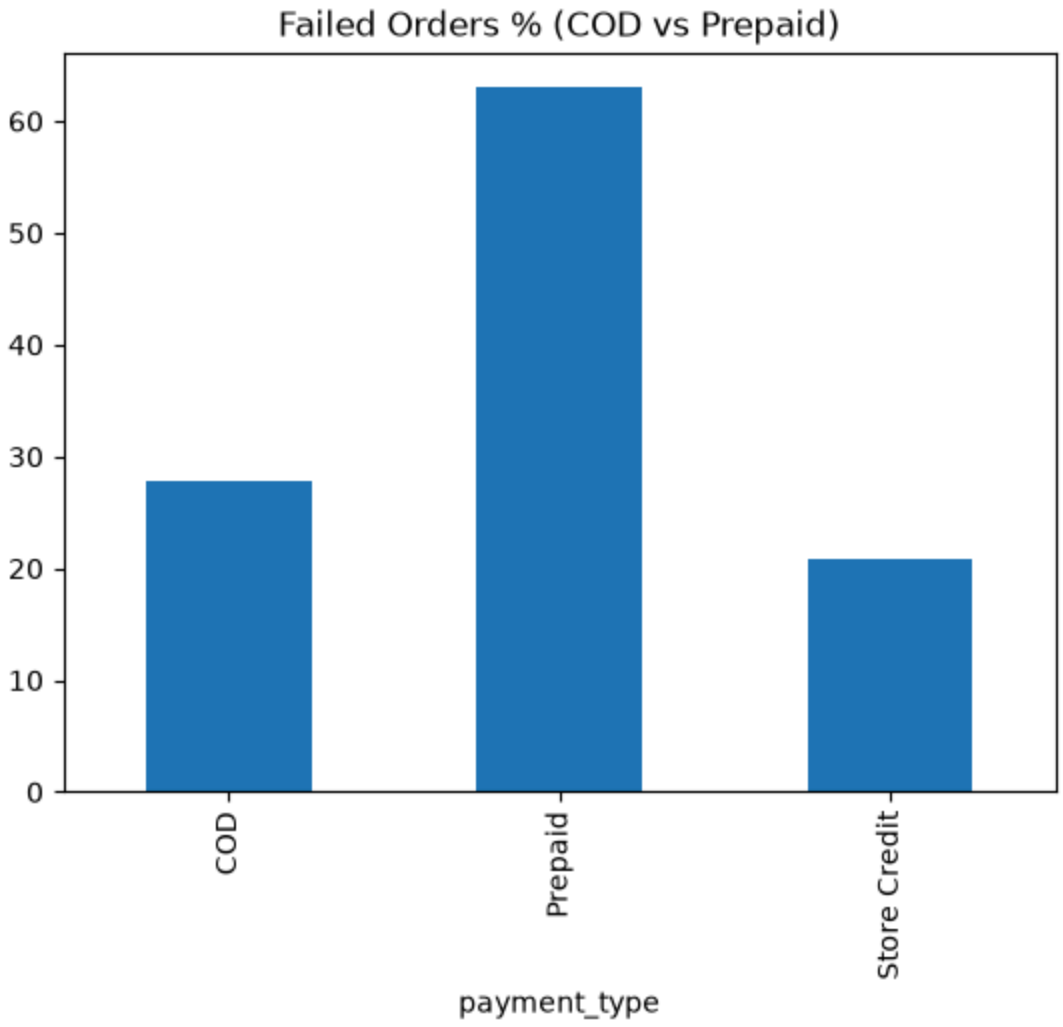
Main question, COD vs Prepaid

In [20]:
```
final = df[df["is_final"] == 1]

result = final.groupby("payment_type")["is_failed"].mean() * 100
print(result.round(2))

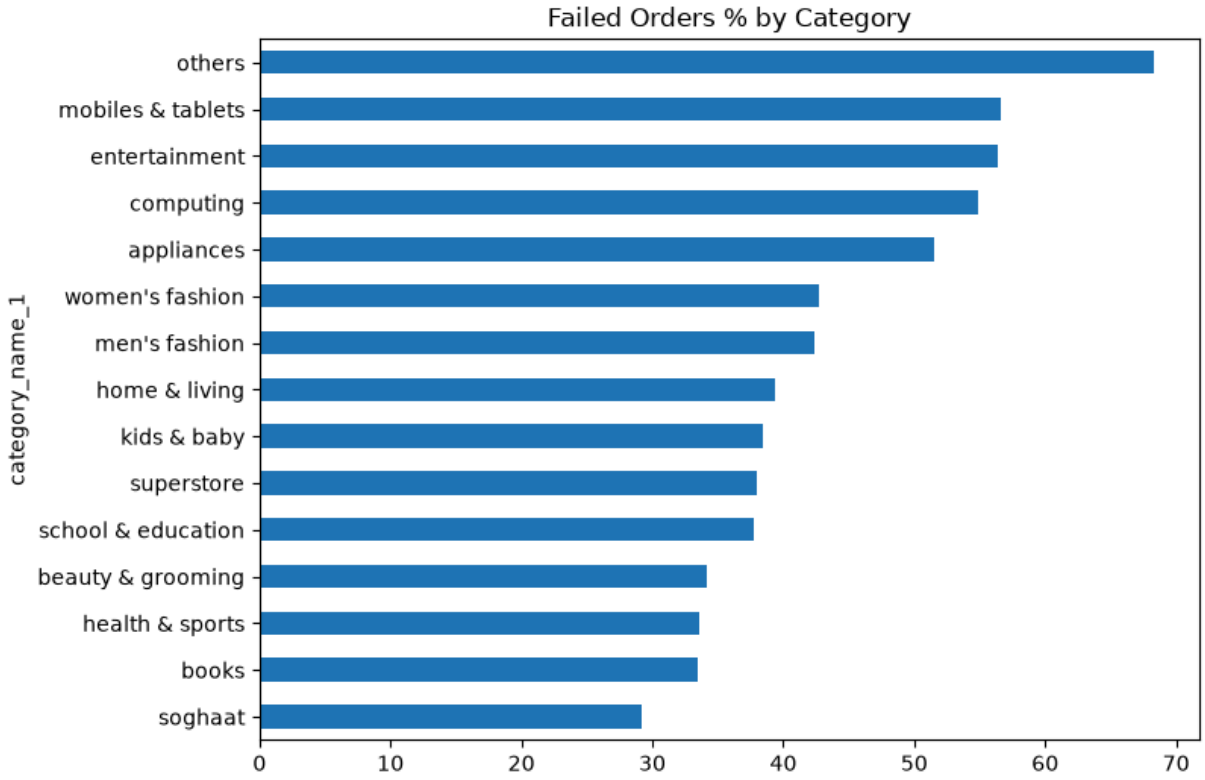
result.plot.bar(title="Failed Orders % (COD vs Prepaid)")
plt.show()
```

```
payment_type
COD             27.84
Prepaid         62.97
Store Credit    20.80
Name: is_failed, dtype: float64
```



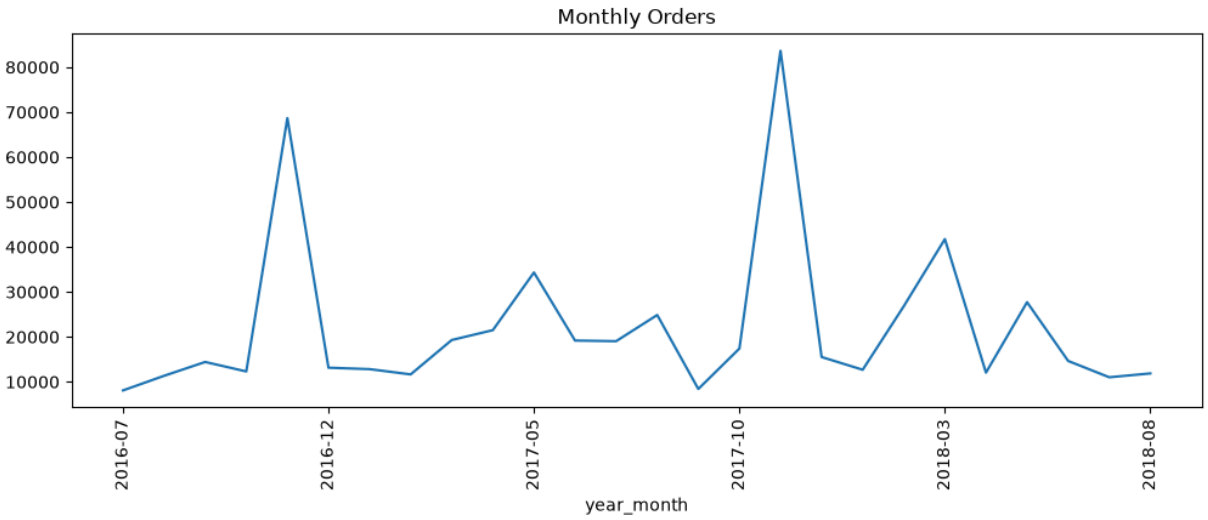
Failed rate by category

In [21]:
```
cat = final.groupby("category_name_1")["is_failed"].mean() * 100
cat.sort_values().plot.barh(figsize=(8, 6), title="Failed Orders % by Category")
plt.show()
```

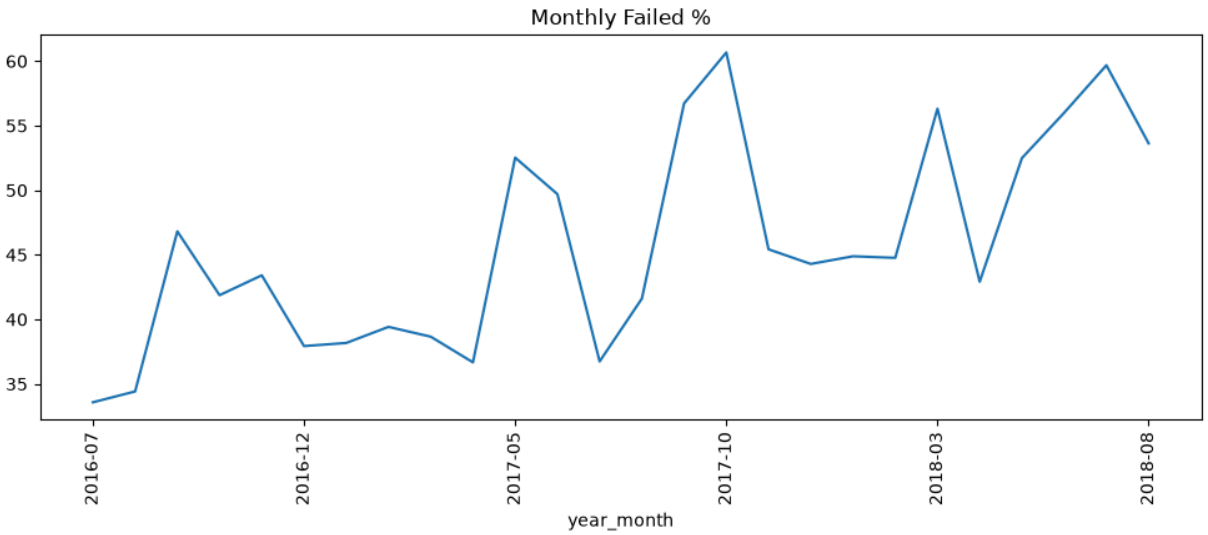


Monthly trend

In [24]:
```
df.groupby("year_month")["order_id"].count().plot(figsize=(12, 4), title="Monthly O
plt.xticks(rotation=90); plt.show()
```

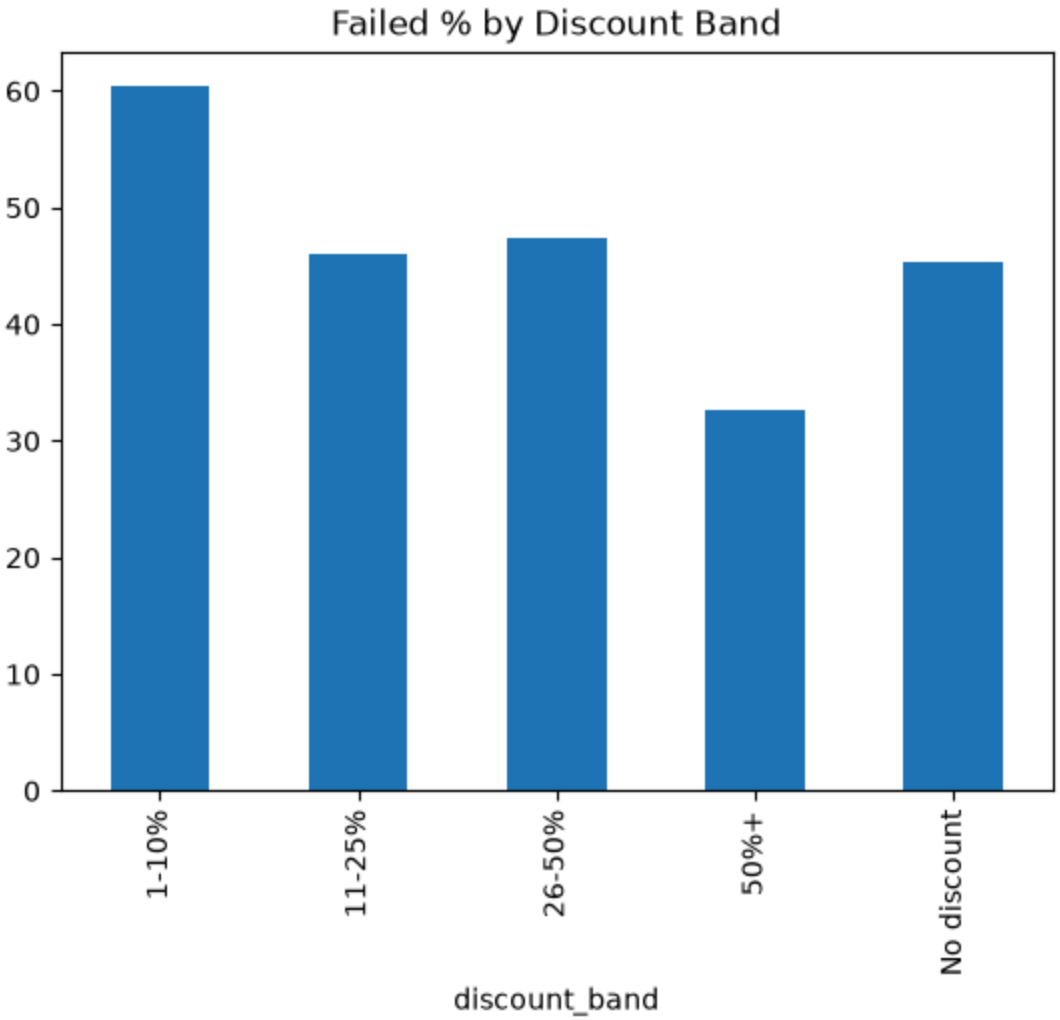


In [25]:
```
(final.groupby("year_month")["is_failed"].mean() * 100).plot(figsize=(12, 4), title
plt.xticks(rotation=90); plt.show()
```

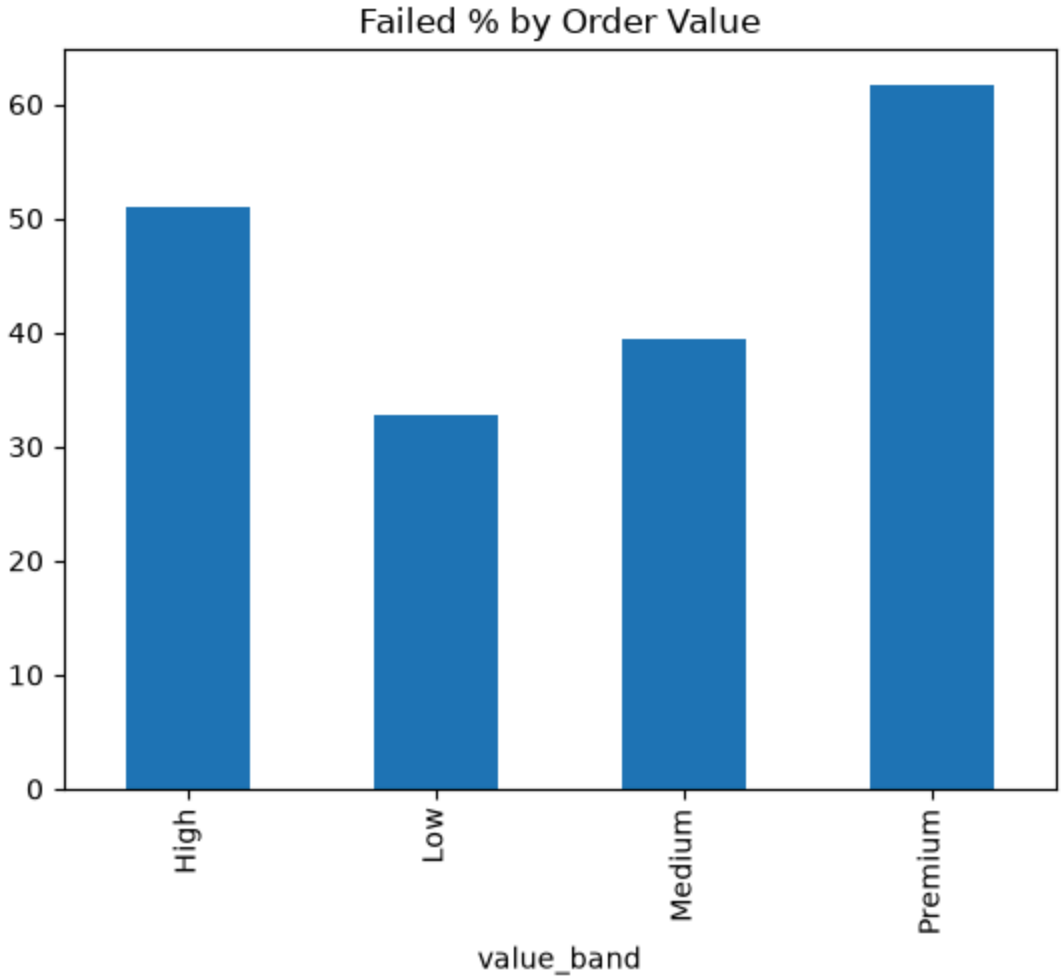


Impact of discount and order value

In [27]:
```
(final.groupby("discount_band")["is_failed"].mean() * 100).plot.bar(title="Failed %
plt.show()
```



In [28]:
```
(final.groupby("value_band")["is_failed"].mean() * 100).plot.bar(title="Failed % by
plt.show()
```



Top categories, gross vs net revenue

In [29]:
```
rev = df.groupby("category_name_1")[["item_total", "net_revenue"]].sum()
rev["lost_%"] = (rev["item_total"] - rev["net_revenue"]) / rev["item_total"] * 100
rev.sort_values("item_total", ascending=False).head(10)
```

Out[29]:

category_name_1	item_total	net_revenue	lost_%
mobiles & tablets	2.307696e+09	6.279122e+08	72.790522
appliances	6.120927e+08	2.333414e+08	61.878093
entertainment	5.277358e+08	2.098166e+08	60.242106
computing	1.856701e+08	4.630189e+07	75.062280
others	1.669594e+08	5.228088e+07	68.686474
women's fashion	1.109939e+08	5.593130e+07	49.608653
men's fashion	8.826006e+07	4.093273e+07	53.622593
beauty & grooming	4.097419e+07	2.153837e+07	47.434305
superstore	4.039331e+07	2.160519e+07	46.512952
home & living	3.227891e+07	1.413795e+07	56.200649

In [ ]: